

A leakage-free feature store for predicting loan default at the point of application

Production style data pipelines on the Medallion Architecture (bronze → silver → gold), built with PySpark and Docker, that prepare the feature store and label store for the model in Assignment 2.

Jaxson Chou

Data Scientist, Consumer Lending

September 2026

Version 1.0

Code

github.com/jiaobuchong/mle-assignment1-dpp

HOW TO READ THIS DECK

Every title states the takeaway and the body carries the evidence, so it reads as a document without a presenter.

2 Executive summary

3 The raw data

4 Architecture

5 Bronze layer

6 Silver layer

7 Gold feature store

8 Leakage controls

9 Validation

10 How to run & next steps

Four raw feeds now flow into one ML-ready feature store and one label store, with every feature restricted to what the bank knew on application day

The pipeline is reproducible from a single command, partitioned by month so it can run as a scheduled job, and validated with a simple model that trains and generalises out of time.

THE PROBLEM

The bank lends cash loans and wants to predict, at the moment of application, whether a customer will default (defined as 30+ days past due six months into the loan). This assignment prepares the data; Assignment 2 trains the model.

WHAT WAS BUILT

- **Bronze:** raw monthly snapshots of the loan ledger, clickstream, customer attributes and financials, landed untouched and partitioned by `snapshot_date`.
- **Silver:** typed and cleaned tables. Corrupted values (placeholders, stray underscores, impossible ages and account counts) become typed columns or explicit nulls.
- **Gold:** a **feature store** (one row per customer per application month, 110 numeric features) and the Lab 2 **label store** (one row per loan, label observed at month 6).

DESIGN DECISIONS THAT MATTER

- **Point-in-time correctness:** clickstream is aggregated over a 6-month window that ends at the application month; the 61% of clickstream rows dated after the application are never read.
- **No silent fixes:** implausible values become null, never a clipped or guessed number.
- **Gold is numeric but un-imputed:** scaling and imputation are model decisions fitted on the training split only, which prevents train-test contamination.
- **One function per table per layer,** same pattern as Lab 2, orchestrated by `main.py` as a monthly backfill.

4 → 10

raw feeds turned into 10 tables across bronze, silver and gold

24

monthly partitions per table, Jan 2023 to Dec 2024, rebuilt in 61 seconds

110

numeric features per customer application, 11,974 rows in the feature store

8,974

labelled loans in the label store, 28.9% default rate, every one matched to features

0.86 AUC

out-of-time sanity model (logistic regression 0.855, gradient boosting 0.870): predictive, and far from the ~1.0 that would signal leakage

Three feature feeds describe the customer at application; the loan ledger is only ever used for labels

Exploratory analysis established the grain of every file, how the files join, and which values cannot be trusted. Two findings shaped the whole design.

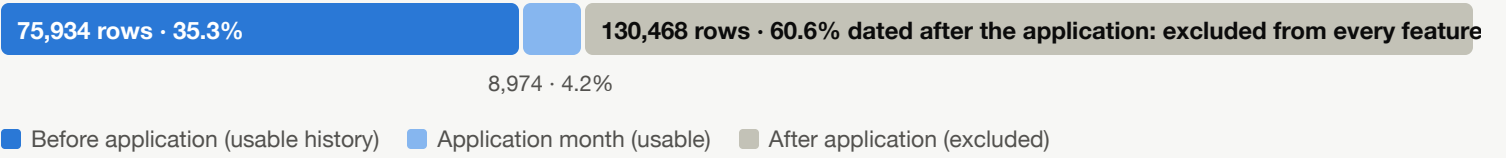
SOURCE FILE	GRAIN	ROWS	COVERAGE	ROLE IN PIPELINE	QUALITY ISSUES OBSERVED IN EDA
lms_loan_daily.csv	loan × month (installment 0 to 10)	137,500	12,500 loans, one per customer, started Jan 2023 to Jan 2025, tracked to Nov 2025	Label store (Lab 2). Never a feature source.	Clean. Loan amount (10,000) and tenure (10) are constant, so they carry no signal.
features_attributes.csv	customer at application	12,500	One row per customer; snapshot_date equals the loan start month for 100% of customers	Demographic features	Age as text with stray underscores and values such as -500 and 8,678; occupation placeholder _____ (7%); SSN garbage #F%D@*&8 (5.6%); name and SSN are PII.
features_financials.csv	customer at application	12,500	Same population and dates as attributes	Financial profile features	52% of rows carry at least one defect: trailing _ on numbers, placeholders __10000__ / _ / !@9#%8 , impossible counts (1,756 bank accounts, 5,789% interest), credit history as text, loan types as a comma list.
feature_clickstream.csv	customer × month	215,376	8,974 customers × 24 months (Jan 2023 to Dec 2024), exactly the customers that receive a label	Behavioural features	Clean values, but 61% of rows are dated after the customer's application: a temporal leakage trap if joined naively.

FINDING 1: THE APPLICATION MONTH IS THE JOIN KEY

Attributes and financials are captured exactly once, in the month the loan starts. The feature store therefore uses Customer_ID + application snapshot_date as its grain, and the label (observed 6 months later) joins back through that date.

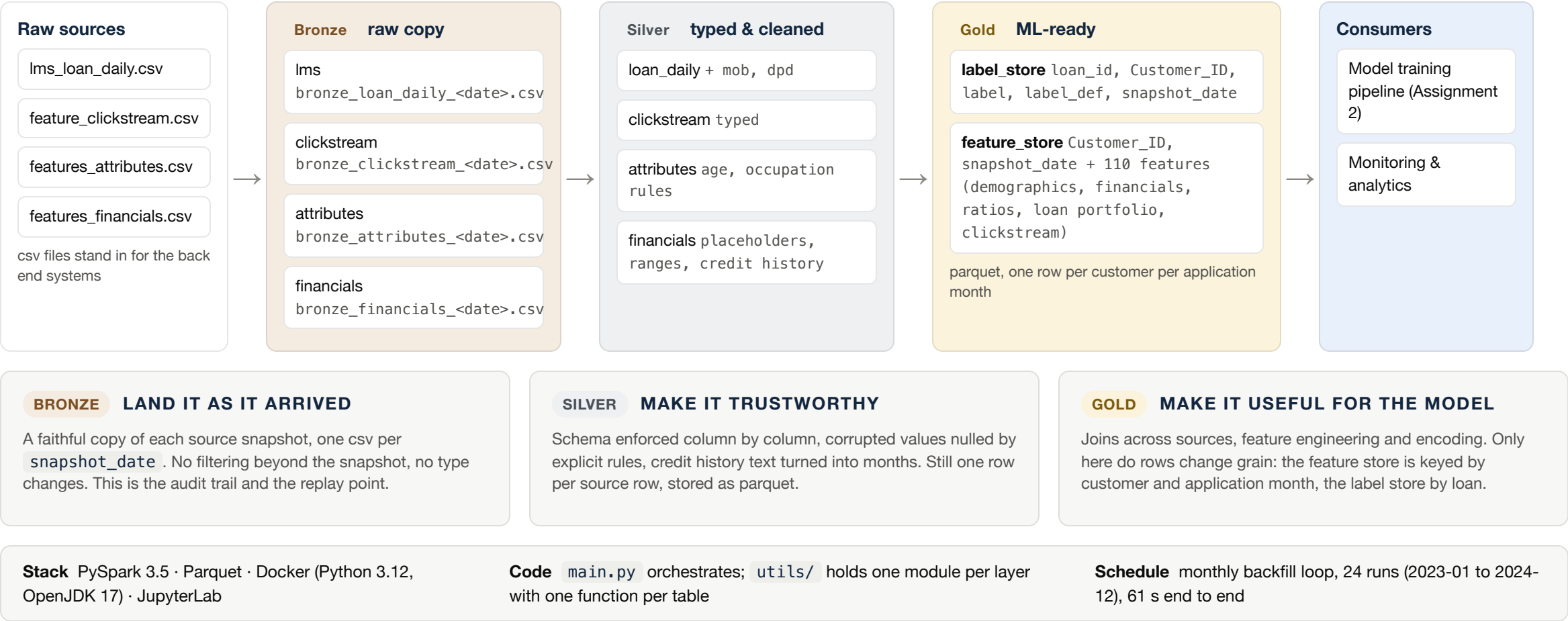
FINDING 2: MOST CLICKSTREAM ROWS ARE FROM THE FUTURE

Clickstream rows by timing relative to each customer's application month (215,376 rows)



Each source moves through bronze, silver and gold as an independent monthly job; the gold layer serves exactly two tables to the model

Ten tables, each partitioned by `snapshot_date` with one file per month. Every partition can be rebuilt from the layer before it, so a bug fix is a re-run, not a manual repair.



Bronze keeps a faithful, replayable copy of every source snapshot, using the same ingestion pattern as Lab 2 for all four feeds

Nothing is interpreted at this stage. If a cleaning rule turns out to be wrong later, silver and gold are rebuilt from bronze without touching the source systems again.

DESIGN CHOICES

- **One partition per `snapshot_date`**. A monthly job reads the source, keeps the rows of that snapshot and writes `bronze_<table>_YYYY_MM_DD.csv`. Backfilling history and running the next month are the same code path.
- **Raw fidelity over convenience**. Values such as `52312.68` or `__10000__` are landed exactly as received; csv keeps them human-readable for investigation.
- **Two small functions, four sources**. The Lab 2 function is kept as is for the loan ledger; a second one takes the table name and csv path and is called three times from `main.py`, so adding a feed is one more call.
- **Source-system boundary**. In production the csv read is the only line that changes: it becomes the database or API connector.

```
# utils/data_processing_bronze_table.py
df = spark.read.csv(csv_file_path, header=True, inferSchema=True) \
    .filter(col("snapshot_date") == snapshot_date)
partition_name = "bronze_" + table_name + "_" + date + ".csv"
df.toPandas().to_csv(bronze_directory + partition_name, index=False)
```

BRONZE TABLES PRODUCED BY `MAIN.PY` (24 MONTHLY PARTITIONS EACH)

TABLE	PARTITION FILE	ROWS / PARTITION	ROWS, 24 MONTHS	NOTE
lms	<code>bronze_loan_daily_<date>.csv</code>	530 → 5,531	104,288	Ledger grows as loans accumulate
clickstream	<code>bronze_clickstream_<date>.csv</code>	8,974	215,376	Same customer set every month
attributes	<code>bronze_attributes_<date>.csv</code>	454 to 543	11,974	That month's applicants
financials	<code>bronze_financials_<date>.csv</code>	454 to 543	11,974	That month's applicants

WHY CSV HERE AND PARQUET LATER

Bronze is optimised for inspection and replay: analysts open a partition in any tool and see the source byte for byte. Silver and gold are optimised for Spark reads and typed schemas, so they switch to parquet. The layout mirrors Lab 2, which makes the label store and the feature store look identical to their consumers.

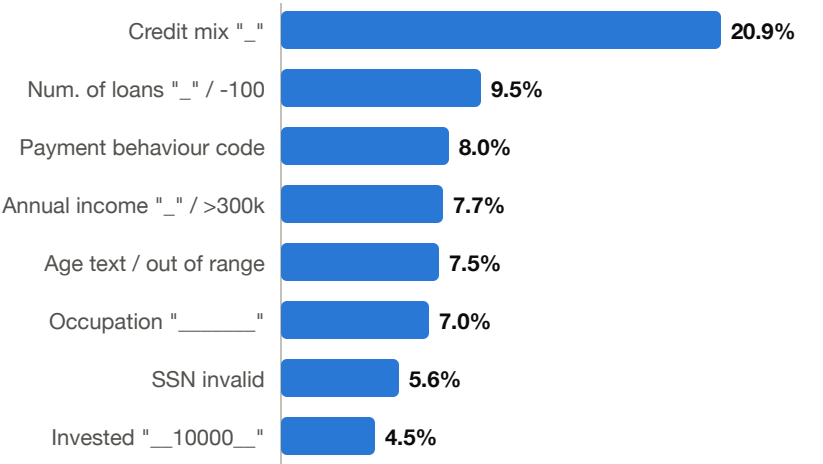
Silver enforces the schema and applies explicit, documented rules so that a corrupted value becomes a null, never a plausible-looking guess

Every rule was derived from the EDA and is written as a whitelist or a plausible range in code, so a reviewer can see exactly what was changed and why.

RULE	RAW EXAMPLE → SILVER	WHY
Schema enforcement	all 4 tables cast column by column	Types are guaranteed downstream; dates become <code>DateTime</code> , counts integers, amounts floats.
Stray underscore	<code>52312.68_</code> → <code>52312.68</code> , <code>3_</code> → <code>3</code>	Formatting noise on otherwise valid numbers (8% of incomes, 5% of loan counts).
Placeholder starting with underscore	<code>_10000_</code> , <code>_</code> → null	A sentinel, not a measurement; keeping 10,000 as an investment amount would be wrong.
Categorical whitelist	<code>_____</code> , <code>_</code> , <code>!@9#%8</code> → null	Occupation, credit mix and payment behaviour only accept their known vocabularies.
Plausible ranges	Age (1 to 100), accounts and cards (0 to 20), interest (0 to 100%), loans (0 to 20), delayed payments and inquiries (0 to 50), EMI ($\leq 5,000$), income ($\leq 300,000$)	Values such as -500 years old, 1,756 bank accounts or 5,789% interest sit far outside a clear gap in the distribution.
Conform text to a number	<code>10 Years and 9 Months</code> → <code>129</code>	Credit history length becomes months, directly usable by a model.
PII left out	Name, SSN (incl. garbage <code>#F%D@*&8</code>)	Kept in silver for lineage only; the gold feature store never reads them.

Share of application rows affected, by issue

12,500 customer applications; a row can carry several defects (52% of financial rows do)



Eight further rules affect 1% to 4% of rows each (EMI, cards, interest, credit limit, inquiries, accounts, debt, balance).

WHY NULL AND NOT A FIX

Clipping 1,756 accounts to 20 or filling an unknown age with the average would present an invented number as fact. Null is honest, is handled natively by gradient boosting models, and lets the model pipeline impute with statistics learned from the training split only.

The gold feature store holds one row per customer per application month with 110 numeric features in six groups, ready to join to the label store

Feature engineering happens only here, from silver inputs, so every feature can be traced back to a cleaned column and a rule.

FEATURE GROUP	COUNT	WHAT IT CAPTURES (EXAMPLES)
Demographics	16	Age , one-hot occupation_* (15 occupations; all zero = unknown)
Financial profile	16	income, salary, bank accounts, credit cards, interest rate, loans, payment delay, delayed payments, credit limit change, inquiries, debt, utilisation, Credit_History_Months , EMI, investment, balance
Affordability ratios	4	debt_to_income , emi_to_salary , invest_to_salary , balance_to_salary
Encoded categoricals	4	credit_mix_score (Bad 0 < Standard 1 < Good 2), pays_min_amount , high_spent , payment_value_size (Small 1 < Medium 2 < Large 3)
Existing loan portfolio	9	one flag per loan type found in Type_of_Loan : auto, credit-builder, debt consolidation, home equity, mortgage, payday, personal, student, not specified (the count itself is Num_of_Loan)
Clickstream behaviour	61	for each of fe_1 to fe_20 : value in the application month, 3-month and 6-month averages over the window ending there; plus the number of months observed

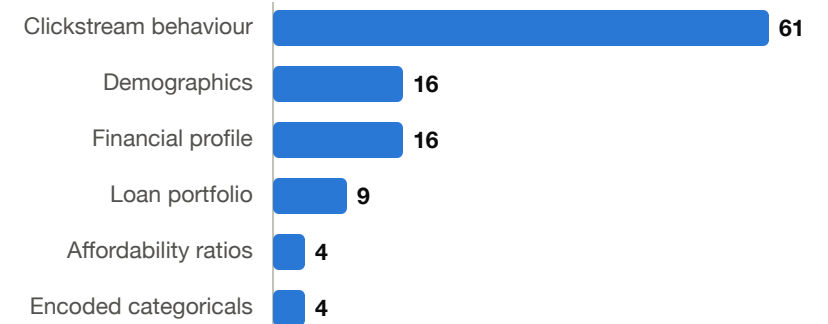
GRAIN AND KEYS

Customer_ID + snapshot_date (application month).
11,974 rows, no duplicate keys, one parquet partition per month: gold_feature_store_YYYY_MM_DD.parquet .

JOIN CONTRACT WITH THE LABEL STORE

feature.Customer_ID = label.Customer_ID and
feature.snapshot_date =
add_months(label.snapshot_date, -6) . The 6 months match the label definition 30dpd_6mob .

Features per group (110 in total)

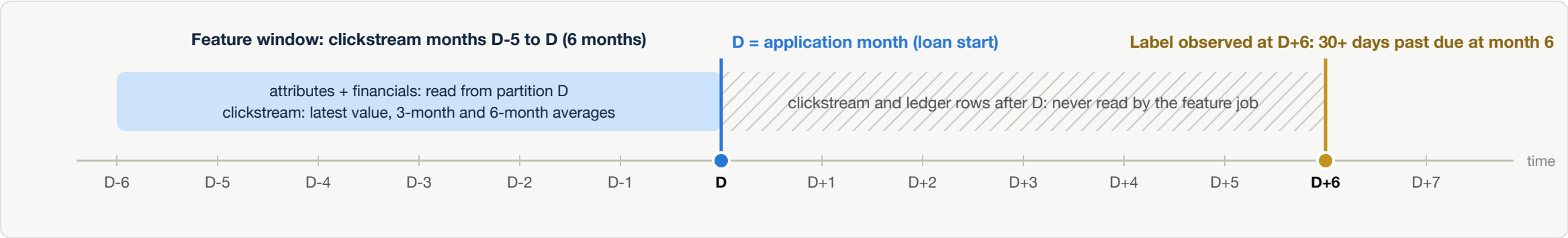


DESIGN PRINCIPLES

- **Numeric only.** Categories are one-hot, ordinal or counted in gold, so any classifier can consume the table directly.
- **Nulls are preserved, nothing is scaled or imputed.** Those steps depend on the training population and belong inside the model pipeline (fit on train, apply to test).
- **Point-in-time by construction.** Attributes and financials are read from the application month's partition; clickstream is filtered to the six months ending there.
- **No PII, no label information.** Name and SSN stop at silver; nothing from the loan ledger enters the feature store.
- **The window is a parameter.** The 6-month look-back is an argument of the gold job, so a longer history is a config change.

Features only see what the bank knew on application day: the window closes at month D, the label is read at month D+6, and nothing in between is used

The three classic leakage paths were each closed by a structural decision in the pipeline rather than by a manual check.



TEMPORAL LEAKAGE

Risk: 61% of clickstream rows are dated after the application, and a naive join on the label's `snapshot_date` would use activity from month D+6.

Control: the gold job keeps only clickstream rows with `0 ≤ months before D < 6`, so nothing dated after D is aggregated. Verified per customer: an applicant of June 2024 has 18 months of history, the window uses the last 6, and the 6 later months are untouched.

TARGET LEAKAGE

Risk: repayment fields (overdue amount, days past due, balance) describe the outcome itself.

Control: the loan ledger feeds only the label pipeline. No column from `lms_loan_daily` exists in the feature store, and the label store carries just the key, the label and its definition.

TRAIN-TEST CONTAMINATION

Risk: scaling, imputing or encoding with statistics of the full dataset leaks the test set into training.

Control: gold stores raw numeric values with nulls; no global statistic is computed in the pipeline. The sanity model splits by application month (train < 2024, test 2024 H1) and fits its imputer and scaler on train only.

The pipeline runs cleanly in Docker, every labelled loan finds its features, and a simple model generalises out of time with an AUC of 0.86

Validation covers the mechanics (does it run, are the counts right), the contract (does the join work) and the substance (are the features predictive without leaking).

PIPELINE RUN (PYTHON MAIN.PY , 61 SECONDS)

LAYER	TABLES	PARTITIONS	ROWS
Bronze	4	96	104,288 ledger · 215,376 clickstream · 11,974 attributes · 11,974 financials
Silver	4	96	same row counts, typed and cleaned, 0 duplicate keys
Gold	2	48	feature store 11,974 × 112 columns · label store 8,974 loans

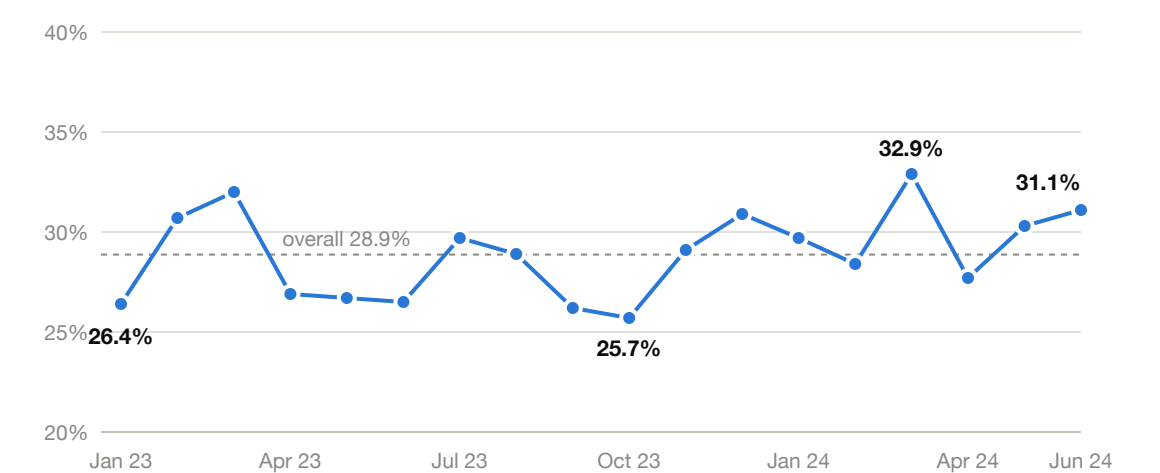
SANITY MODEL (PYTHON SANITY_CHECK_MODEL.PY , NOT PART OF THE PIPELINE)

CHECK	RESULT
Label rows matched to features via the join contract	8,974 of 8,974 , all with clickstream history
Out-of-time split by application month	train 5,958 (Jan to Dec 2023) · test 3,016 (Jan to Jun 2024)
Logistic regression (median impute + scale, fit on train)	train AUC 0.852 · test AUC 0.855
Gradient boosting (nulls handled natively)	train AUC 0.941 · test AUC 0.870

Test AUC close to train AUC and well below 1.0 is the signature of an informative, leakage-free feature store. Strongest linear drivers: salary, income, 6-month clickstream averages, interest rate. Tuning is left to Assignment 2.

Default rate by application month (label store, 8,974 loans)

Share of loans 30+ days past due at month 6. The base rate is stable around 29%, so the label is usable for training without re-weighting.



WHAT WAS CHECKED BY HAND

- Clickstream aggregates recomputed from the raw file for a sample customer match the gold values exactly (latest, 3-month and 6-month means).
- Occupation flags never exceed one per row; loan type flags match the text in `Type_of_Loan` ; cleaned ages span 14 to 56, incomes stop at 180k.

Two commands reproduce the whole datamart; the same modules are ready to be scheduled month by month and consumed by the model pipeline

Everything runs inside the provided container, so reviewers need Docker and nothing else.

RUN IT

```
# 1. build and start the environment (JupyterLab on http://localhost:8888)
docker-compose build
docker-compose up

# 2. in a JupyterLab terminal: build bronze, silver and gold (about 1 minute)
python main.py

# 3. optional: join feature and label store, fit the sanity model
python sanity_check_model.py
```

WHAT MAIN.PY CREATES

```
datamart/
  bronze/ lms/ clickstream/ attributes/ financials/      24 csv each
  silver/ loan_daily/ clickstream/ attributes/ financials/ 24 parquet each
  gold/   label_store/ feature_store/                    24 parquet each
```

SUBMISSION CONTENTS

`main.py` · `Dockerfile` · `docker-compose.yaml` · `requirements.txt` · `utils/` (bronze, silver, gold modules) · `data/` · `Readme.txt` (repository link) · this deck as PDF. The `datamart/` folder is generated and excluded.

Repository: github.com/jiaobuchong/mle-assignment1-dpp

NEXT STEPS

Assignment 2: model training pipeline.

Read both gold tables, apply the join contract, split by application month, fit imputation and scaling on train only, and register the model with its feature list and label definition.

Schedule the monthly jobs.

Each layer function already takes a single `snapshot_date` ; wrapping them as tasks in an orchestrator (for example Airflow) turns the backfill loop into a scheduled DAG with one run per month.

Data quality monitoring.

Log row counts, null shares and the share of rows hit by each silver rule per partition, and alert when a month drifts (for example a jump in placeholder values or in the clickstream averages).

Widen the feature store when the feeds allow.

Attributes and financials are captured once per customer today; if the source systems refresh them monthly, the same gold job can serve every customer every month, enabling re-scoring beyond the application moment.